Лабораторная работа №1
«Вероятностные алгоритмы»

Примечание:

Когда в тексте написано «оценить процент…/погрешность» имеется в виду, что для конкретного набора (например, k, m, n и % наполненности) эксперимент должен быть запущен некоторое количество раз. Потом результат (процент или погрешность) усредняются средним арифметическим. Плюсом будет узнать выборочную дисперсию. Чем больше количество раз, тем надежнее результаты. Минимальное количество раз примем за три запуска.

Задание №1

Обозначение:

k – количество хеш-функций,

m – размер массива,

n – предполагаемый максимум для множества,

ε – вероятность ложноположительных срабатываний.

1. Реализовать фильтр Блума (класс). Выбор реализации хеш-функции зависит от вашего варианта (таблица 1). Значение хеш-функции ограничить до 2^{30}. Учесть, чтобы была возможность (т. е. опционально) при создании экземпляра значение k и m автоматически высчитывать в зависимости от передаваемых необязательных параметров (ε и n) в экземпляр по формулам 2 и 3.
2. Реализовать фильтр Блума со счетом с использованием реализации из 1-го пункта.
3. Написать юнит-тесты и оценить процент ложноположительных срабатываний конкретных реализаций (фильтр Блума со счетом) и сравнить с теоретическим значением (формула 1). n и ε выбрать в зависимости от первой буквы фамилии (таблица 2), k и m рассчитать по формулам 2 и 3 соответственно. Наполненность множества на 25%, 50%, 75% и 95% от числа n. При оценке необходимо проверить данными, которых нет, в количестве 100% от наполненности множества. (Т. е. если n = 15 000, множество наполнилось на 25%, и было добавлено 3 750 данных. Тогда, необходимое количество данных, которые не были добавлены – 3 750).
4. Оценить зависимость между параметрами структуры и процентом ложноположительных срабатываний (графики и таблица).
5. При реализации учесть возможность пересечения и объединения фильтров Блума через перегрузку операторов «+» (*dunder add*) и «–» (*dunder*

sub), если используется *Python*/*C++*/*C#*, или создание публичных методов, реализующих данный функционал.

6. Произвести тестирование (юнит-тесты) операций пересечение и объединения полученных структур из 4 пункта (операции объединения и пересечения должны быть между структурами с одинаковыми параметрами и хеш-функциями). Проверить наличие и отсутствие элементов, которые должны быть в полученном множестве, и элементов, которые не должны быть, соответственно.

Таблица 1 – Реализация хеш-функции по вариантам

Вариант	1	2	3	4
реализация	*MD5*	*SHA-1*	*SHA-256*	*SHA3-256*
модуль	*hashlib*	*hashlib*	*hashlib*	*hashlib*
Вариант	5	6	7	
реализация	*MurmurHash3*	*CityHash64*	*BLAKE2b*	
модуль	*mmh3*	*cityhash*	*hashlib*	

Таблица 2 – Параметры для юнит-тестов по вариантам

Диапазон первой буквы фамилии	[А, …, Й]		
	1	2	3
n	15 000	100 000	500 000
ε	0.5	0.2	0.05
Диапазон первой буквы фамилии	[К, …, Ф]		
	1	2	3
n	25 000	130 000	750 000
ε	0.7	0.15	0.05
Диапазон первой буквы фамилии	[Х, …, Я]		
	1	2	3
n	30 000	75 000	400 000
ε	0.6	0.25	0.05

Формула расчёта ошибки:

$$\varepsilon = \left(1 - e^{\frac{-kn}{m}}\right)^k. \quad (1)$$

Формула расчета оптимального значения k:

$$k = \frac{m}{n} \ln 2. \quad (2)$$

Формула оптимального значения m:

$$m = -\frac{n \ln \varepsilon}{\ln^2 2}. \quad (3)$$

Задание №2

Обозначение:

n – предполагаемый максимум для множества,

p – параметр (количество бит, определяющих разбиение на подмножества),

q – размер регистров/счетчиков (обычно 32),

ε – процентная погрешность.

1. Реализовать *HyperLogLog* (класс). Выбор из таблицы 1 реализации хеш-функции зависит от значения v_2 (новый вариант).

Данное значение рассчитывается по формуле: $v_2 = (v_1 + 1)\%6 + 1$, где v_1 – вариант из предыдущего задания, операция % – остаток от деления.

Учесть, чтобы была возможность (т. е. опционально) при создании экземпляра значение p автоматически высчитывать в зависимости от передаваемого необязательного параметра (ε) в экземпляр по формуле 4.

2. При реализации учесть возможность объединения структур через перегрузку операторов «+» (*dunder add*), если используется *Python*/*C++*/*C#*, или создание публичных методов, реализующих данный функционал.

3. При реализации воспользоваться эвристиками для просчета кардинального числа на малых и очень больших мульти множествах (см. справку).

4. Реализовать потоковую бесконечную генерацию случайных данных (например дат) через генераторы, если используется *Python*.

5. Оценить погрешность (в абсолютных и процентных величинах) конкретных трех реализациях на разных потоках данных. Размер потока смотреть в таблице 3. Данный пункт реализовать в юнит-тестах.

6. Оценить погрешность структуры, полученной в результате объединения предыдущих реализаций для *small* и *norm*. Данный пункт реализовать в юнит-тестах.

7. Оценить зависимость погрешности от параметров алгоритма (таблица и графики зависимостей).

Таблица 3 – размер потока по вариантам

Вариант	1			2			3			4		
	small	norm	big	small	norm	big	small	norm	big	small	norm	big
Размер потока	25 000	200 000	1 000 000	15 000	300 000	900 000	10 000	250 000	1 150 000	20 000	150 000	1 500 000
Вариант	5			6			7					
	small	norm	big	small	norm	big	small	norm	big			
Размер потока	21 000	241 000	1 000 111	18 000	310 000	999 999	12 000	279 000	1 150 460			

Справка:

$$p = \left\lceil 2\log_2\left(\frac{1.04}{\varepsilon}\right)\right\rceil \tag{4}$$

Оценка уникальных элементов:

$$E = a_m m^2 Z.$$

Значение $m = 2^{\mathrm{p}}$.

Значение a_m может быть вычислено двумя способами.

1-й способ (не рекомендуется):

$$a_m = \left(m \int_0^{\infty} \left(\log_2 \left(\frac{2+u}{1+u} \right) \right)^m \, du \right)^{-1}.$$

2-й способ:

$$a_m = \begin{cases} 0.673, & m = 16; \\ 0.697, & m = 32; \\ 0.709, & m = 64; \\ \dfrac{0.7213}{1 + \dfrac{1.079}{m}}, & m \geq 128. \end{cases}$$

Значение $Z = \left(\sum_{j=1}^{m} 2^{-M[j]} \right)^{-1}$, где M – массив счетчиков, длинной m.

Что делать при оценке малых мульти множеств:

В данном случае используют алгоритм *linear counting*.

Если много нулевых регистров (обозначим V— число регистров с $M[j] = 0$), то это сигнал, что элементов мало, и лучше работает оценка:

$$\hat{n}_{small} = m \ln \left(\frac{m}{V} \right). \tag{5}$$

Формулой (5) необходимо воспользоваться при выполнении двух условий: $V > 0$, $E < \frac{5}{2} m$.

Что делать при оценке очень больших мульти множеств:

При очень больших мощностях, приближающихся к пределу размера регистров ($\mathrm{E} > \frac{2^q}{30}$) мощность множества можно оценить следующим образом:

$$\hat{n}_{big} = -2^{32} \log \left(1 - \frac{\mathrm{E}}{2^{32}} \right).$$

Задание №3

Обозначение:

d – число хеш-функций,

w – ограничение сверху возвращаемого хеш-функциями значения,

C_{dw} – матрица счетчиков,

ε – масштаб допустимой ошибки,

δ – параметр надёжности.

1. Реализовать *Count-Min Sketch* (класс). Выбор из таблицы 1 реализации хеш-функции зависит от значения v_3 (новый вариант).

Данное значение рассчитывается по формуле: $v_3 = (v_2 + 5)\%6 + 1$, где v_2 – вариант из предыдущего задания, операция % – остаток от деления. Учесть, чтобы была возможность (т. е. опционально) при создании экземпляра значение d и w автоматически высчитывать в зависимости от передаваемых параметров (ε и δ) в экземпляр по формулам 6.

2. Оценить погрешность конкретных двух реализаций с помощью генератора, реализованного в задании №2 п.4. Размер потока выбрать по вариантам из таблицы 3 (*norm* и *big*). Данный пункт реализовать в юнит-тестах.

3. Оценить погрешность структуры, полученной в результате объединения предыдущих двух реализаций. Данный пункт реализовать в юнит-тестах.

4. Оценить зависимость погрешности (абсолютной и относительной) оценки от гиперпараметров алгоритма (таблица и графики зависимостей).

Оптимальный выбор параметров:

$$w = \left\lceil \frac{e}{\varepsilon} \right\rceil, d = \left\lceil \ln \frac{1}{\delta} \right\rceil \tag{6}$$